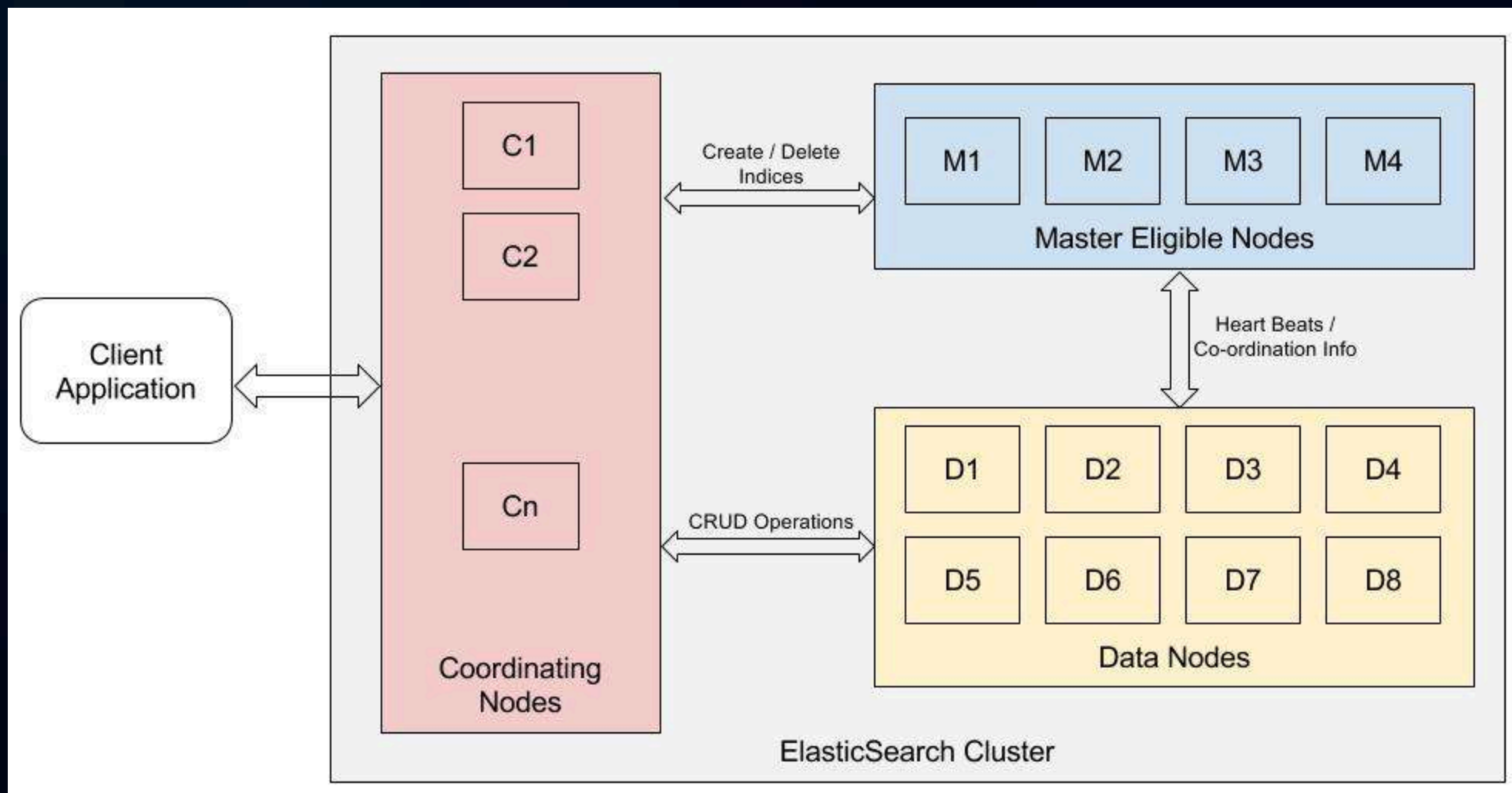


Mastering Elasticsearch

Architecture, Optimization, and Stability Strategies for High-
Performance Distributed Clusters
at Production Scale

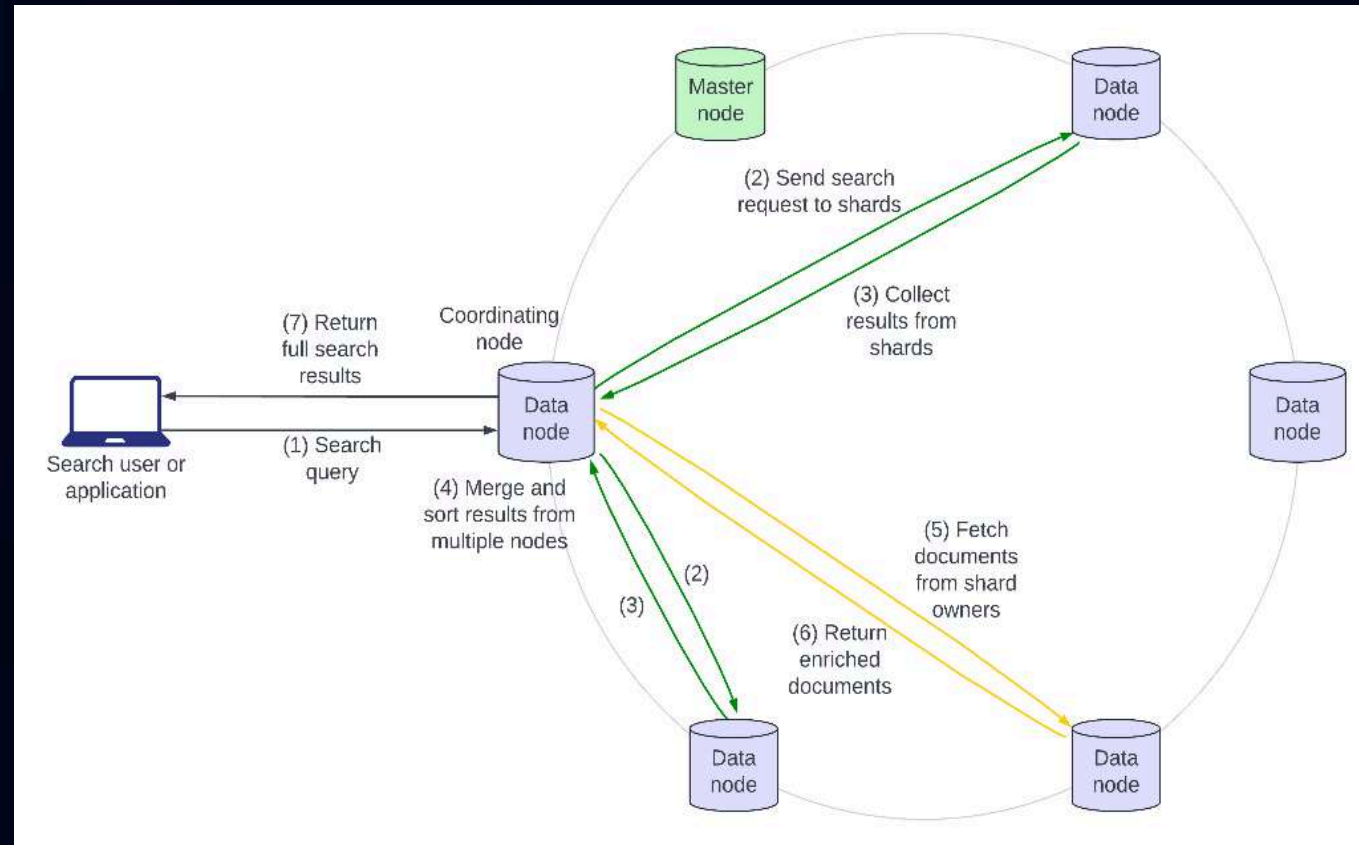
Multi Node Architecture



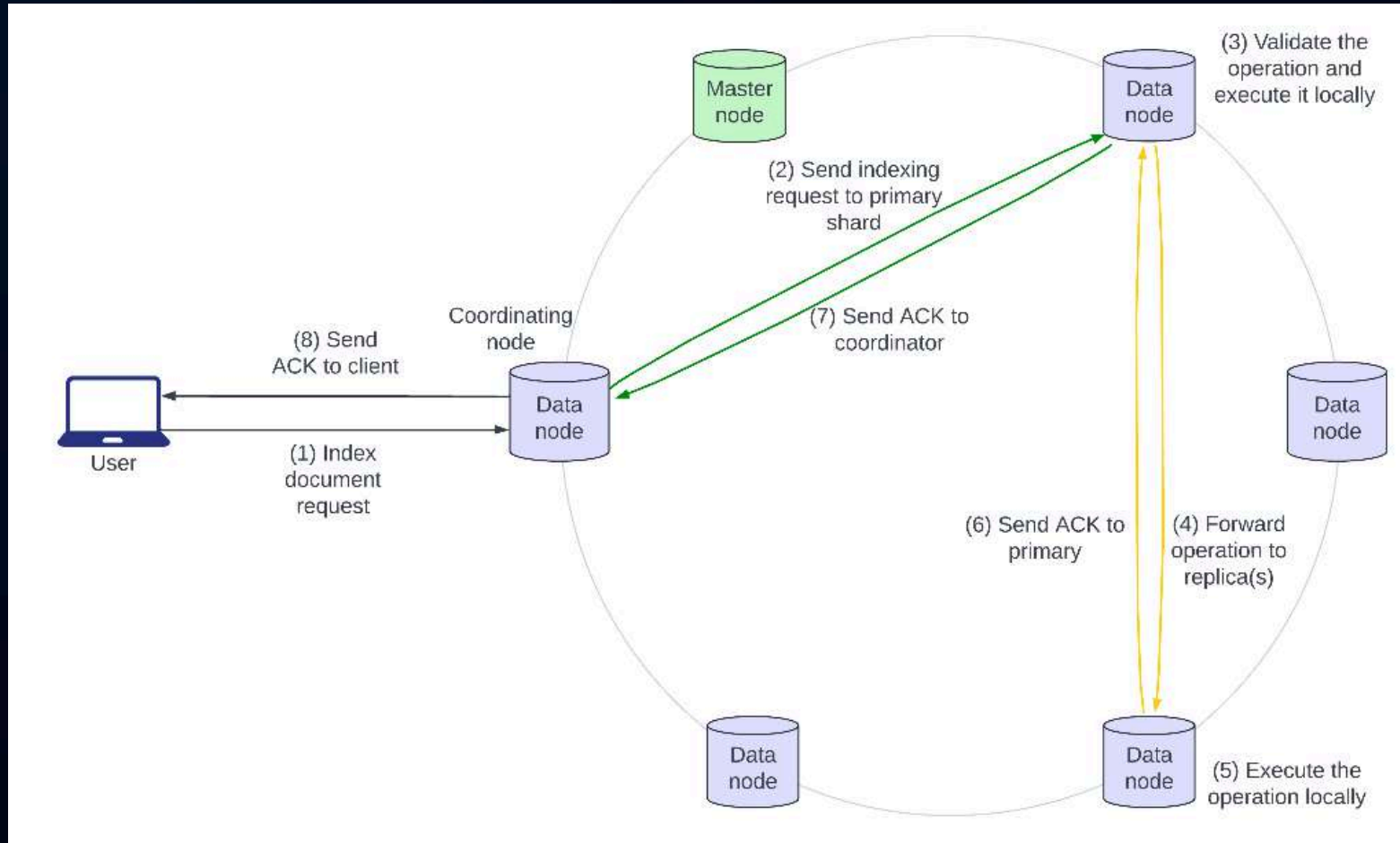
Search: Query & Fetch Phases

1. Query Phase: The coordinating node broadcasts the request to every shard. Shards execute locally, returning only **IDs and Scores**. Memory overhead is low as no data moves.

2. Fetch Phase: After sorting global top hits, the node requests full **_source** data only from shards containing the winners. This "Scatter-Gather" pattern saves bandwidth.



Indexing Flow



| The Shard Sizing "Golden Rule"



10GB - 50GB

Aim for shard sizes in this range. Small shards increase overhead; large shards (over 50GB) make recovery and relocation painfully slow.



Heap Ratio

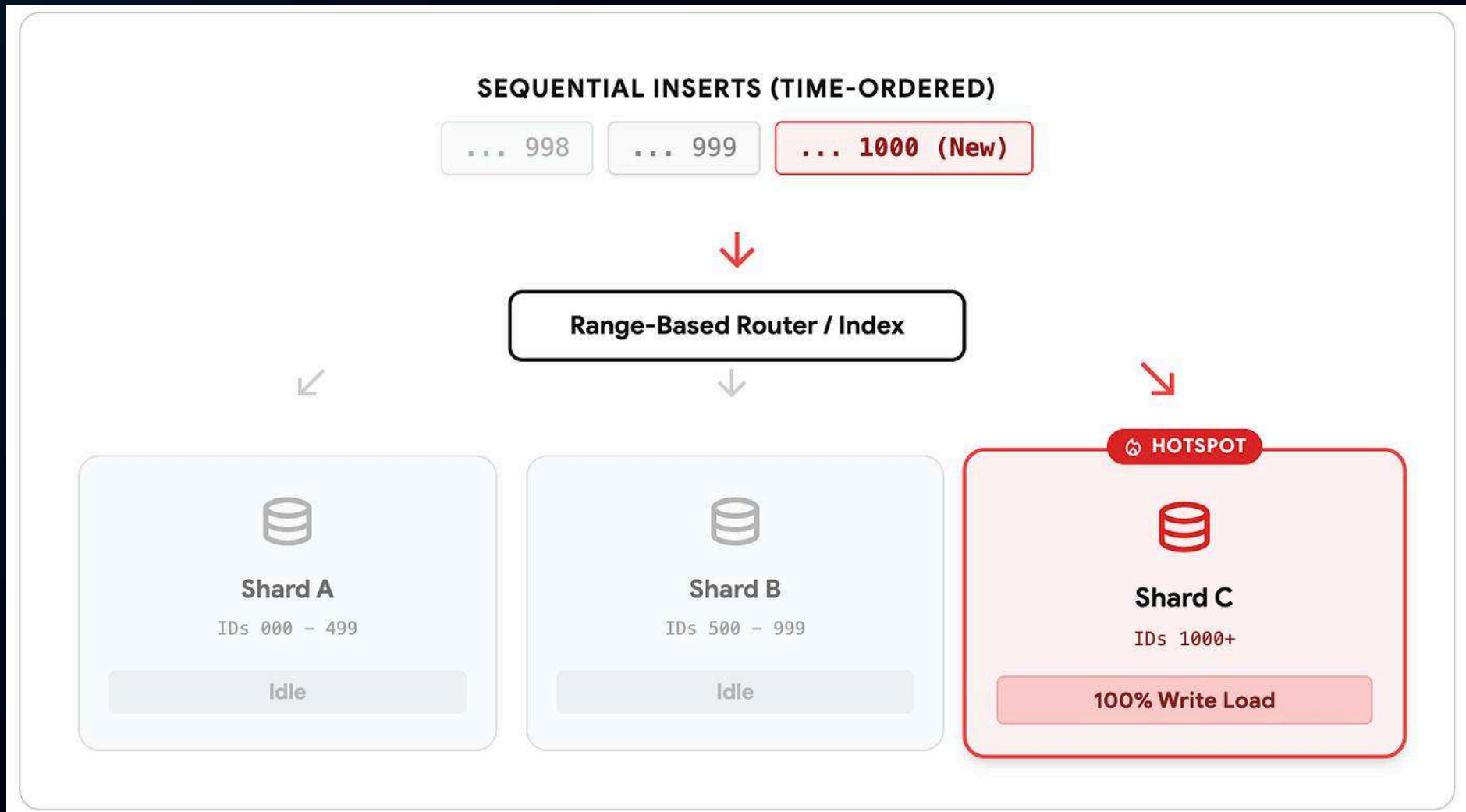
Keep a ratio of roughly **20 shards per GB** of heap. Over-sharding leads to cluster-state bloat and master node instability.



Doc Limits

Avoid exceeding **200 million documents** per shard to maintain search efficiency and merge performance within Lucene.

Hotspots





Resolving Hotspots

ROLLOVER API

Shift from age-based to size-based indexing.

Roll over when a shard hits **50GB**, not just at midnight to avoid uneven shard sizes.

SPLIT / SHRINK

`_split`: Increase primary shards for hot indices.

`_shrink`: Reduce shards in Warm tiers to optimize storage.

HOT THREADS

Use `_nodes/hot_threads`

Identify the exact code path causing high CPU on a specific hot node in real-time.

REPLICA STRATEGY

Scale for Read Hotspots.

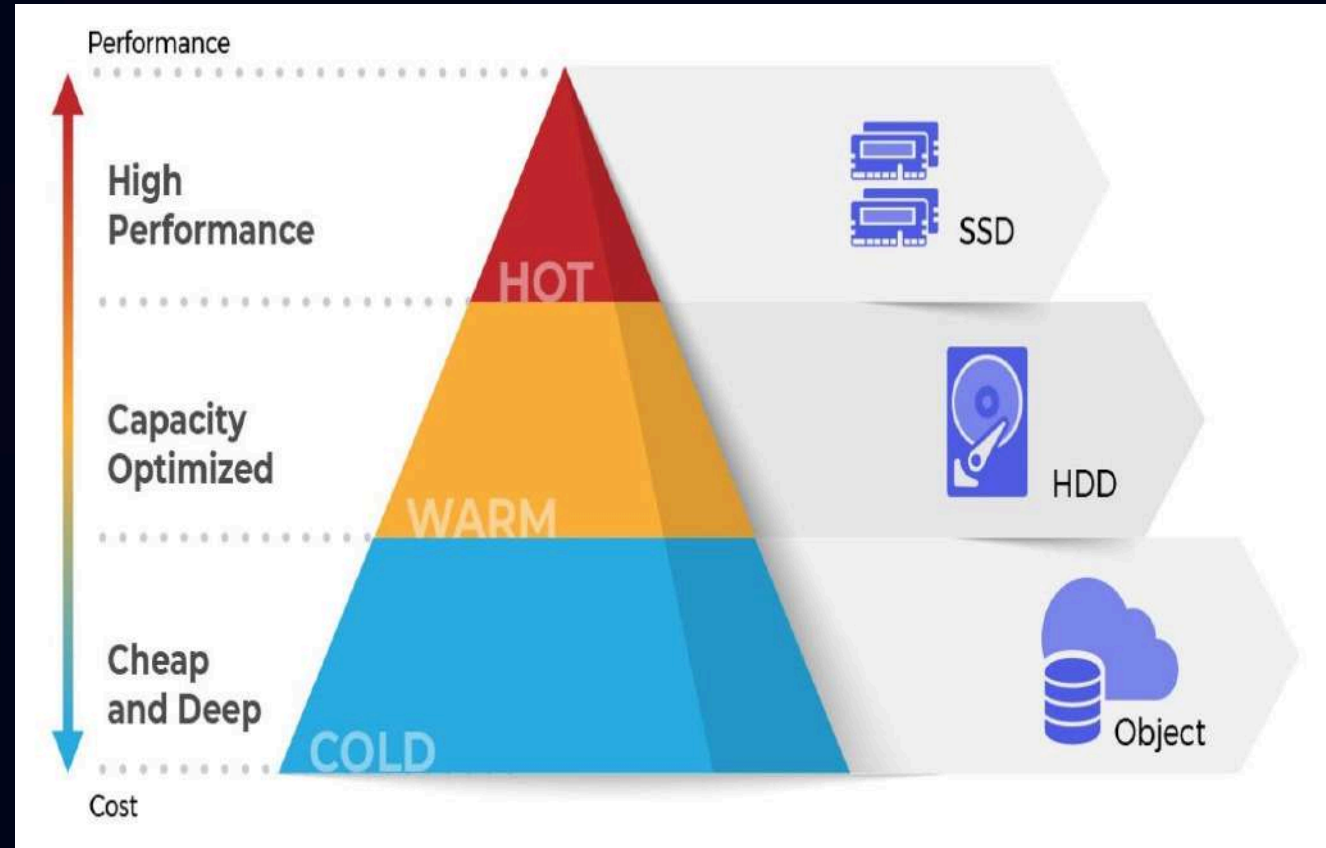
Increase replicas to distribute query load across more nodes and improve search throughput.

Index Design & Lifecycle (ILM)

Hot Phase: Fast SSDs. Handles high-ingestion and primary search traffic.

Warm Phase: Larger HDDs. Read-only indices. Use **Shrink** to reduce shard count and **Force Merge** for search speed.

Cold/Frozen: Cheapest storage. Searchable snapshots for long-term retention with low cost.



Index Design & Lifecycle (ILM)

HOT PHASE

```
// High Performance
"hot": {
  "actions": {
    "rollover": {
      "max_size": "25gb",
      "max_age": "1d"
    },
    "set_priority": {
      "priority": 100
    }
  }
}
```

WARM PHASE

```
// Capacity Optimized
"warm": {
  "min_age": "2d",
  "actions": {
    "shrink": {
      "num_shards": 1
    },
    "forcemerge": {
      "max_segments": 1
    },
    "set_priority": {
      "priority": 50
    }
  }
}
```

COLD PHASE

```
// Cheap & Long-term
"cold": {
  "min_age": "30d",
  "actions": {
    "searchable_snapshot": {
      "snapshot_repository": "r1"
    },
    "set_priority": {
      "priority": 0
    }
  }
}
```

| Latency Optimization & Caching

1. Filtering vs Query (The Efficiency Gap)

Expensive (Query Context)

```
{ "query": { "term": { "status": "ACTIVE" } } }
```

- Computes **score**
- No caching benefit (in most cases)

Efficient (Filter Context)

```
"bool": { "filter": [ { "term": { "status": "ACTIVE" } } ] }
```

- No scoring → **skips BM25**
- Uses **bitsets (cached)**
- Reusable across queries

2. Caching Layers (Critical for Scale)

Query Cache

Per Shard. Caches filter results.

Best when:

- Same filters repeat
- Static data

Request Cache

Whole Response. Identical queries.

Great for:

- Dashboards
- Aggregations

OS Page Cache

The Hidden Hero. Linux filesystem.

Outcome:

- Fast fetch phase
- Data in memory

Query Tuning for Speed

1. Filter First, Then Score

Bad (everything scored)

```
{ "query": { "match": { "title": "..." } } }
```

Good (filter → then score)

```
"bool": { "filter": [...], "must": [...] } }
```

Filter reduces dataset first; scoring runs on smaller set.

2. match vs term

Slower (match)

```
"match": { "user": "john" }
```

Faster (term)

```
"term": { "user.keyword": "john" }
```

'term' is a direct lookup. Use keyword fields for exact matches.

3. Control Scoring

Ranking doesn't matter? Disable it.

- **constant_score:** Same score for all; skip calculation cost.
- **Filter Context:** No BM25 calculation → huge speedup.

Stop overpaying for scores you don't use.

4. Limit Returned Data

Bad: "_source": true

Good: "_source": ["title", "price"]

Reducing returned fields significantly decreases:

- Disk read operations
- Network payload latency

5. Avoid Expensive Queries

Wildcard: Very slow. Use **prefix** or **edge_ngram**.

Scripts: Runs per document. Extremely slow.

Perform costly logic at index-time, not query-time.

Bulk API: Efficiency vs. Overhead

Single-Doc Indexing

Inefficient for high-volume data.

- 1 HTTP request per document
- Heavy network + parsing overhead
- No batching → Poor throughput

Bulk Indexing

Amortised costs across documents.

- Many docs in **one request**
- Reduced network cost
- Less thread scheduling
- Optimized disk I/O

Example: `_bulk` request format

```
{ "index" : { "_index" : "test", "_id" : "1" } }  
{ "field1" : "value1" }  
{ "delete" : { "_index" : "test", "_id" : "2" } }  
{ "create" : { "_index" : "test", "_id" : "3" } }  
{ "field1" : "value3" }
```

Bulk API: Key Optimization Levers

1. Batch Size

Ideal Target:

- 5MB – 15MB per request
- ~1k – 10k documents

Benchmark your specific workload for best results.

2. Parallelism

Bulk Workers:

Run multiple workers to fully utilize CPU, I/O, and Network resources.

Watch for thread pool rejection if too high.

3. Intervals

Refresh Settings:

Increase **refresh_interval** (e.g., 30s) during heavy ingestion to boost throughput.

Higher interval = Better performance.

4. Replicas

Temporary Toggle:

Set **number_of_replicas: 0** during ingestion to avoid duplicate writes.

Re-enable (set to 1) once complete.

5. Shards

Write Parallelism:

Increase primary shard count for hot ingestion to allow more parallel writes.

Avoid over-sharding (overhead).

6. Backpressure

Handling 429s:

When the cluster is overloaded, use **Exponential Backoff** in your client.

Retry slowly instead of failing.

Common Production Failures

1. Heap Pressure

Symptoms

- High latency spikes
- Frequent GC pauses
- Node drops from cluster

Why

- Too many shards
- Large aggregations / bulk

Debug & Fix

`GET _nodes/stats/jvm`

- Reduce shard count
- Use keyword for aggs

2. Slow Queries

Symptoms

- Latency 500ms → 5s
- High CPU on some nodes
- User complaints > Alerts

Why

- Wildcards & Script queries
- Poor mappings

Debug & Fix

`GET _slowlog`

- Replace wildcard with ngram
- Move filters to filter context

3. Hotspot Shards

Symptoms

- Single node CPU = 90%
- Others idle
- Uneven latency

Why

- Skewed data (dominant user)
- Latest shard in time-series

Debug & Fix

`GET _cat/shards?v`

- Increase shard count
- Use rollover indices

Common Production Failures

4. Thread Pool Rejections

Symptoms

- "status": 429
- Requests failing under load
- Bulk indexing failures

Why

- High concurrency
- Pools saturated

Debug & Fix

`GET _cat/thread_pool?v`

- Reduce concurrency
- Tune bulk size

5. Disk Watermarks

Symptoms

- Indexing stops
- No shard relocation
- Cluster yellow/red

Why

Thresholds: 85% (low), 90% (high), 95% (flood)

Debug & Fix

`GET _cat/allocation?v`

- Delete old indices
- Add storage / Use ILM

6. Unassigned Shards

Symptoms

- Health: Red/Yellow
- Indices unavailable

Why

- Node failure
- Allocation blocks

Debug & Fix

`GET _cluster/allocation/explain`

- Add nodes
- Fix filters

7. Mapping Explosion

Symptoms

- Heap creep
- Slow queries

Why

Dynamic fields (user_ID_field) creating thousands of mappings.

Debug & Fix

`GET index/_mapping`

- Disable dynamic mapping
- Flatten structure

Questions?

Ensuring Elasticsearch remains stable at scale requires constant monitoring of heap, disk, and shard distribution.